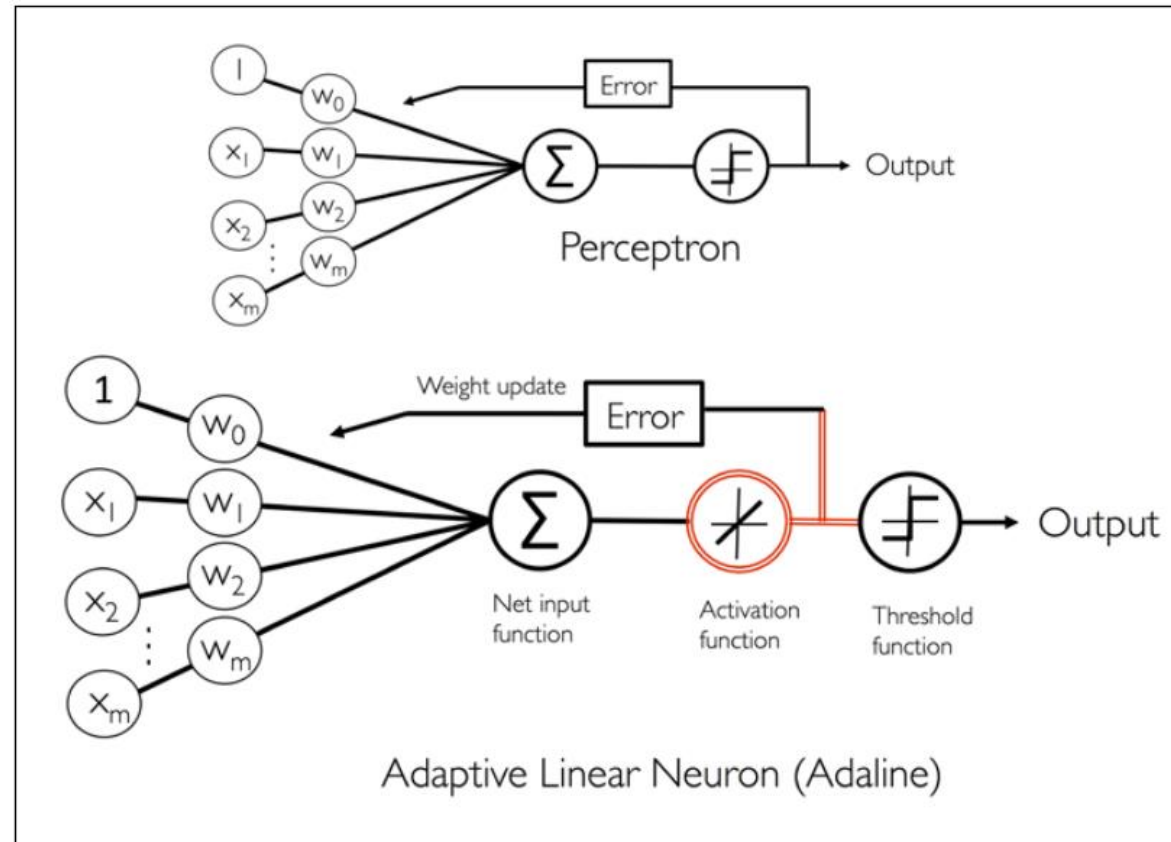


Perceptron

❁ A **perceptron** is one of the **simplest** types of artificial neural networks and a fundamental **building block** of deep learning.

❁ It was introduced by Frank Rosenblatt in 1957 as a binary classifier.



What is its assumption

A linear function (boundary)

How it works?

Suppose, we have a sample with feature value $(x_1, x_2, \dots, x_m)$, the predict class for the sample is:

$$z = \begin{cases} 1, & \text{if } w_0 + w_1x_1 + w_2x_2 + \dots + w_mx_m \geq 0 \\ -1, & \text{if } w_0 + w_1x_1 + w_2x_2 + \dots + w_mx_m < 0 \end{cases}$$

Where the parameters $(w_0, w_1, w_2, \dots, w_m)$ come from?

The parameters are learned/trained using a bunch of historical data.

Suppose, we have n samples:

Sample 1: $(x_1^1, x_2^1, \dots, x_m^1)$ with class y^1

Sample 2: $(x_1^2, x_2^2, \dots, x_m^2)$ with class y^2

...

Sample n: $(x_1^n, x_2^n, \dots, x_m^n)$ with class y^n

Input

1. Randomize $(w_0, w_1, w_2, \dots, w_m)$

2. For each sample i in $(1, 2, \dots, n)$:
$$\bar{y} = \begin{cases} 1, & \text{if } w_0 + w_1x_1^i + w_2x_2^i + \dots + w_mx_m^i \geq 0 \\ -1, & \text{if } w_0 + w_1x_1^i + w_2x_2^i + \dots + w_mx_m^i < 0 \end{cases}$$
$$w_0 = w_0 + \eta(y^i - \bar{y})$$
$$w_1 = w_1 + \eta(y^i - \bar{y})x_1^i$$
$$w_2 = w_2 + \eta(y^i - \bar{y})x_2^i$$
$$\dots$$
$$w_m = w_m + \eta(y^i - \bar{y})x_m^i$$

3. Repeat step 2 for each epoch until the error falls below a threshold or the maximum number of epochs is reached.

Iteration rule

$(w_0, w_1, w_2, \dots, w_m)$

Output

Advantage:

- 1.Simple to understand
- 2.Easy to implement

Disadvantage:

- 1.Can only used for linearly separable data
- 2.It will never converge for non-linearly separable data. So it's better to decide the number of maximum iteration.
- 3.For loop is used for each sample. So it may take a lot of time for data with large amount of samples.

An example from real data

Iris data

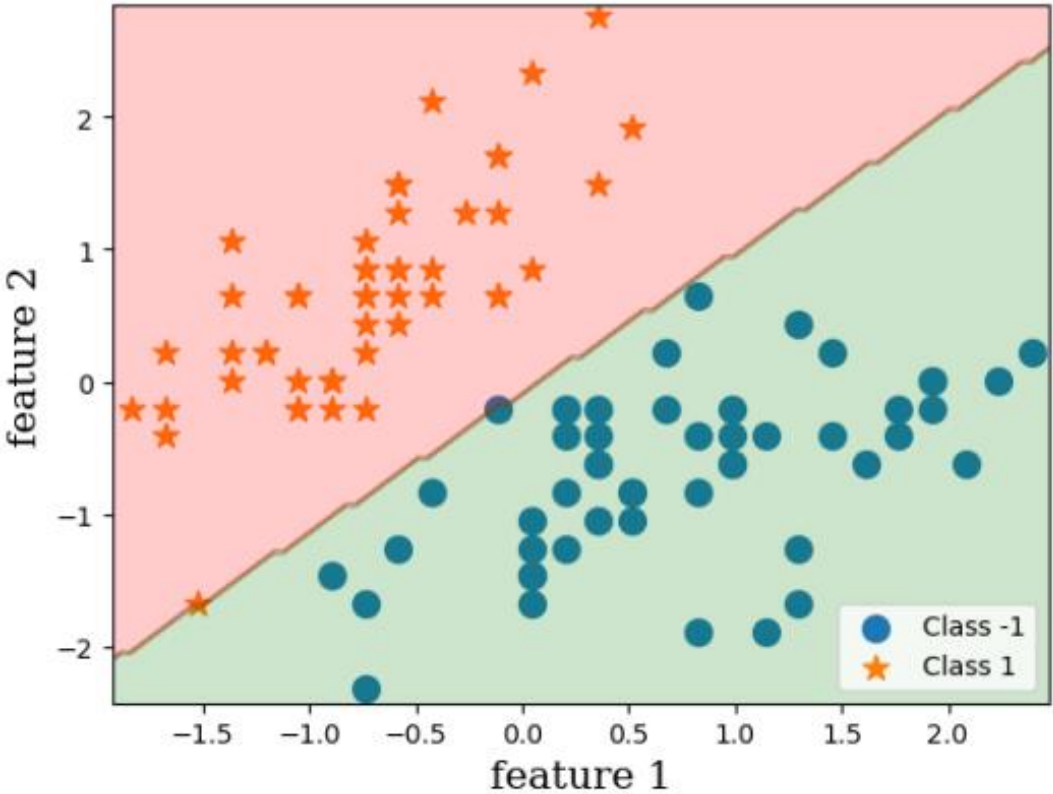
It includes three iris species with 50 samples each as well as some properties about each flower.

The columns in this dataset are:

Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species

	Index	0	1	2	3	4
	0	5.1	3.5	1.4	0.2	Iris-setosa
	1	4.9	3	1.4	0.2	Iris-setosa
	2	4.7	3.2	1.3	0.2	Iris-setosa
	3	4.6	3.1	1.5	0.2	Iris-setosa
	4	5	3.6	1.4	0.2	Iris-setosa
	5	5.4	3.9	1.7	0.4	Iris-setosa
	6	4.6	3.4	1.4	0.3	Iris-setosa
	7	5	3.4	1.5	0.2	Iris-setosa

Iris
data



The classification outcome

Python code

1

```
import numpy as np
class Perceptron():
    def __init__(self, eta):
        self.w=[]
        self.w0=[]
        self.eta=eta
        self.error=[]
    def net_input(self,X):
        return np.matmul(X,self.w)+self.w0
    def predict(self,X):
        z=self.net_input(X)
        return np.where(z>0,1,-1)
    def fit(self,X,y):
        self.w=np.random.random(X.shape[1])
        self.w0=np.random.random(1)[0]
        for epoch in range(100):
            c=np.hstack([X,y.reshape(X.shape[0],-1)])
            np.random.shuffle(c)
            X,y=c[:,0:2],c[:,2]
            k=0
            for i in range(X.shape[0]):
                y_hat=self.predict(X[i,:])
                self.w=self.w+self.eta*(y[i]-y_hat)*X[i,:]
                self.w0=self.w0+self.eta*(y[i]-y_hat)
                if y[i]!=y_hat:k=k+1
            k=k/X.shape[0]
            self.error.append(k)
```

2

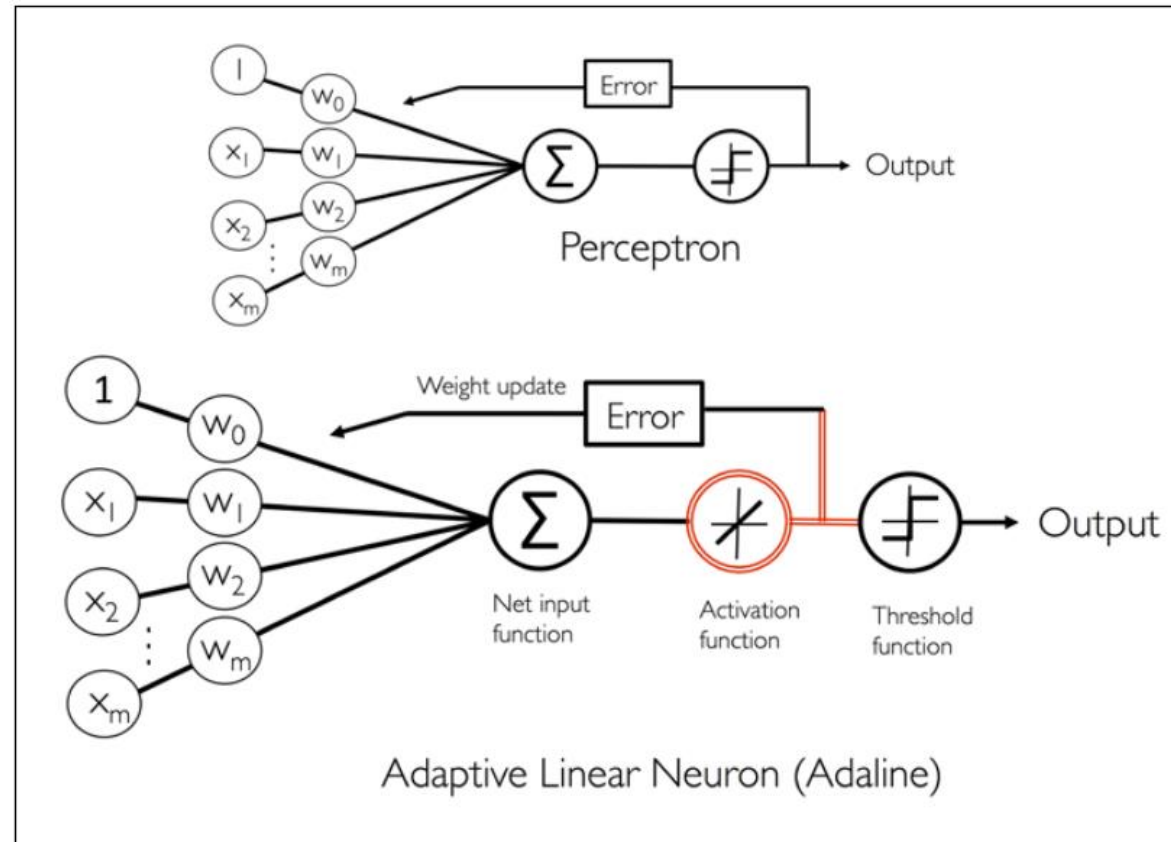
```
def plot_boundary_graph(X,y,classifier):
    font2 = {'family':'serif','color':'black','size':15}
    label=np.unique(y)
    x1=X[(y==label[0])]
    plt.scatter(x1[:,0],x1[:,1],s=100,marker='o',label=f'Class {label[0]}')
    x1=X[(y==label[1])]
    plt.scatter(x1[:,0],x1[:,1],s=100,marker='*',label=f'Class {label[1]}')
    plt.legend(loc='lower right')
    plt.xlabel('feature 1', fontdict = font2)#('sepal length')
    plt.ylabel('feature 2', fontdict = font2)#('petal length')
    x0=np.arange(min(X[:,0])-0.05*abs(min(X[:,0])),max(X[:,0])+0.05*abs(min(X[:,0])),(max(X[:,0])-min(X[:,0]))/100)
    y0=np.arange(min(X[:,1])-0.05*abs(min(X[:,1])),max(X[:,1])+0.05*abs(min(X[:,1])),(max(X[:,1])-min(X[:,1]))/100)
    x01,y01=np.meshgrid(x0,y0)
    c=np.hstack((x01.reshape(-1,1),y01.reshape(-1,1)))
    c_y=classifier.predict(c)
    plt.contourf(x01,y01,c_y.reshape(x01.shape),alpha=0.2,colors=['g','g','r','r'])
    plt.show()
```

3

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df=pd.read_csv(r'F:\learning\ML and DL with Python\ch02\iris.data',header=None)
a=pd.unique(df[4])
df.loc[df[4]==a[0],4]=1#df.loc[row_indexer, "col"] = values
df.loc[df[4]==a[1],4]=-1
df=df[0:100]
# shuffle the DataFrame rows
df = df.sample(frac = 1)
X=df.iloc[:,0:2].values
y=df.iloc[:,4].values
#normalization
X=(X-X.mean(axis=0))/X.std(axis=0)
f=Perceptron(0.001)
f.fit(X,y)
plot_boundary_graph(X,y,f)
```

Adaline

- ❁ The Adaline (Adaptive Linear Neuron) algorithm is an single-layer neural network model.
- ❁ It was introduced by Bernard Widrow and Ted Hoff in 1960 as a binary classifier.
- ❁ It uses a linear activation function with gradient descent-based learning rule.



Key Features of Adaline:

1.Linear Activation Function: Unlike the Perceptron, which uses a step function (threshold function) for classification, Adaline uses a **linear activation function** for learning weights but applies a threshold for prediction.

2.Continuous Output: The output is computed as: $z = W^T X + b$

3.Mean Squared Error (MSE) Loss: Instead of minimizing misclassifications directly, Adaline minimizes the **sum of squared errors (SSE)** between the predicted output and true label:

4.Gradient Descent Optimization: Weights are updated using gradient descent to minimize the loss

m features, n samples

$$X = \begin{bmatrix} x_1^1 & x_2^1 & \cdots & x_m^1 \\ x_1^2 & x_2^2 & \cdots & x_m^2 \\ \cdots & \cdots & \cdots & \cdots \\ x_1^n & x_2^n & \cdots & x_m^n \end{bmatrix} \quad Y = \begin{bmatrix} y^1 \\ y^2 \\ \cdots \\ y^n \end{bmatrix}$$

Net input function for sample i :

$$z^i = \sum_{j=1}^m x_j^i w_j + b$$

Activation function for sample i :

$$\varphi(z^i) = z^i$$

Loss function for all samples:
sum of squared errors (SSE)

$$J(w) = \frac{1}{2} \sum_{i=1}^n (y^i - \varphi(z^i))^2$$

How to get Parameters W and b

$$W = \begin{bmatrix} w_1 \\ w_2 \\ \cdots \\ w_m \end{bmatrix} \quad b$$

Partial derivative for parameter w_j :

$$\frac{\partial J}{\partial w_j} = - \sum_{i=1}^n \left(y^i - \left(\sum_{j=1}^m x_j^i w_j + b \right) \right) x_j^i$$

1. Initialize W and b randomly

2. Update W and b (gradient descent)

The iteration rule for W and b :

$$W = W + \Delta W = W - \eta \nabla J(W)$$

$$W = W + [(Y - XW)^T X]^T$$

$$b = b + \text{sum}(Y - XW)$$

3. Repeat step 2 until the error falls below a threshold or the maximum number of epochs is reached.

4. Get W and b

$$\text{Gradient: } \nabla J(W) = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \frac{\partial J}{\partial w_2} \\ \cdots \\ \frac{\partial J}{\partial w_m} \end{bmatrix}$$

η is the learning rate.
It is a hyperparameter.

epoch is a hyperparameter.

An example from real data

Iris data

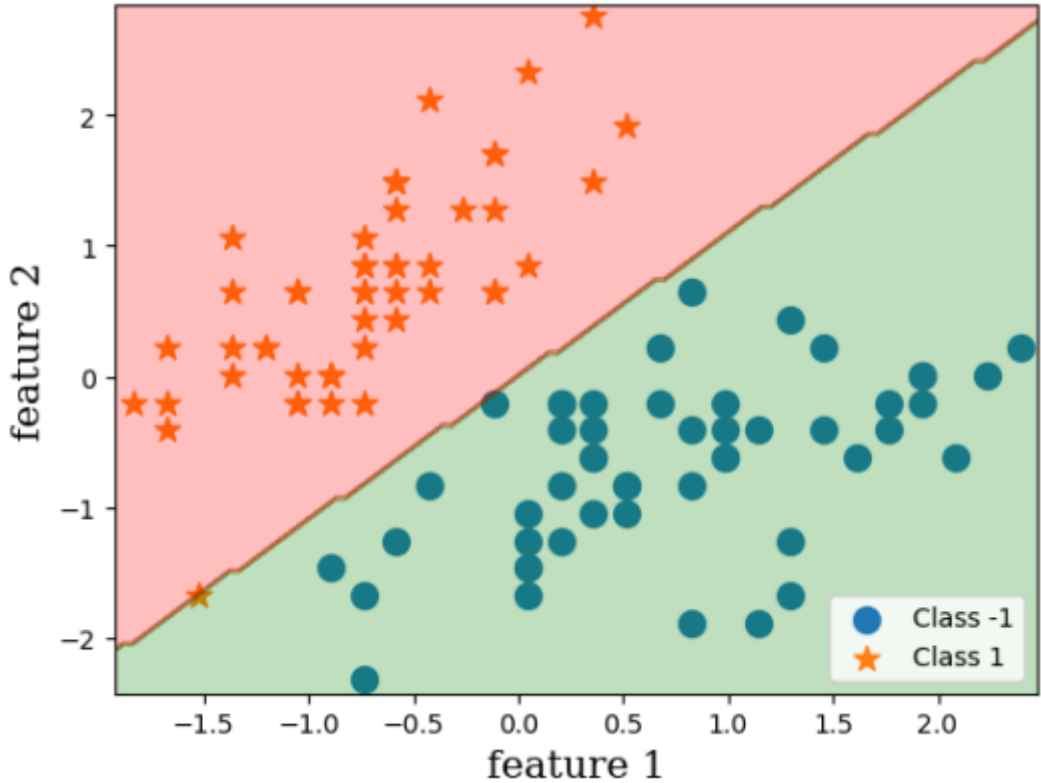
It includes three iris species with 50 samples each as well as some properties about each flower.

The columns in this dataset are:

Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species

	Index	0	1	2	3	4
	0	5.1	3.5	1.4	0.2	Iris-setosa
	1	4.9	3	1.4	0.2	Iris-setosa
	2	4.7	3.2	1.3	0.2	Iris-setosa
	3	4.6	3.1	1.5	0.2	Iris-setosa
	4	5	3.6	1.4	0.2	Iris-setosa
	5	5.4	3.9	1.7	0.4	Iris-setosa
	6	4.6	3.4	1.4	0.3	Iris-setosa
	7	5	3.4	1.5	0.2	Iris-setosa

Iris
data



The classification outcome

2

```
def plot_boundary_graph(X,y,classifier):
    font2 = {'family':'serif','color':'black','size':15}
    label=np.unique(y)
    x1=X[(y==label[0])]
    plt.scatter(x1[:,0],x1[:,1],s=100,marker='o',label=f'Class {label[0]}')
    x1=X[(y==label[1])]
    plt.scatter(x1[:,0],x1[:,1],s=100,marker='*',label=f'Class {label[1]}')
    plt.legend(loc='lower right')
    plt.xlabel('feature 1', fontdict = font2)#('sepal length')
    plt.ylabel('feature 2', fontdict = font2)#('petal length')
    x0=np.arange(min(X[:,0])-0.05*abs(min(X[:,0])),max(X[:,0])+0.05*abs(min(X[:,0])),(max(X[:,0])
    y0=np.arange(min(X[:,1])-0.05*abs(min(X[:,1])),max(X[:,1])+0.05*abs(min(X[:,1])),(max(X[:,1])
    x01,y01=np.meshgrid(x0,y0)
    c=np.hstack((x01.reshape(-1,1),y01.reshape(-1,1)))
    c_y=classifier.predict(c)
    plt.contourf(x01,y01,c_y.reshape(x01.shape),alpha=0.25,colors=['g','g','r','r'])
    plt.show()
```

1

```
class Adaline():
    def __init__(self,eta):
        self.w=[]
        self.w0=[]
        self.eta=eta
        self.error=[]
    def net_input(self,X):
        return np.matmul(X,self.w)+self.w0
    def activation(self,X):
        return X
    def predict(self,X):
        z=self.net_input(X)
        return np.where(z>0,1,-1)
    def fit(self,X,y):
        self.w=np.random.random(X.shape[1])
        self.w0=np.random.random(1)[0]
        for epoch in range(100):
            y_hat=self.net_input(X)
            y_hat=self.activation(y_hat)
            er=sum((y-y_hat)**2)
            if er<0.0001:
                break
            else:
                self.error.append(er)
                self.w=self.w+self.eta*np.matmul((y-y_hat),X)
                self.w0=self.w0+self.eta*sum(y-y_hat)
```

3

```
import pandas as pd
import numpy as np
import numpy as npexit
import matplotlib.pyplot as plt
df=pd.read_csv(r'F:\Learning\ML and DL with Python\ch02\iris.data',header=None)
a=pd.unique(df[4])
df.loc[df[4]==a[0],4]=1#df.loc[row_indexer, "col"] = values
df.loc[df[4]==a[1],4]=-1
df=df[0:100]
# shuffle the DataFrame rows
df = df.sample(frac = 1)
X=df.iloc[:,0:2].values
y=df.iloc[:,4].values
#normalization
X=(X-X.mean(axis=0))/X.std(axis=0)
f=Adaline(0.001)
f.fit(X,y)
plot_boundary_graph(X,y,f)
```

Note:

1. Instead of updating the weights after evaluating each individual training sample, as in the perceptron, we calculate the gradient based on the whole training dataset
2. We can see how the information flows through a single layer neural network: features from the input data, net input, activation, and output.
3. Gradient descent is one of the many algorithms that benefit from feature scaling.

Gradient descent

The parameters are updated once for the whole data set

```
elif optimizer=='GD':  
    y_hat=self.net_input(X)  
    y_hat=self.activation(y_hat)  
    er=sum((y-y_hat)**2)  
    self.w=self.w+self.eta*np.matmul((y-y_hat),X)  
    self.w0=self.w0+self.eta*sum(y-y_hat)
```

Mini-batch gradient descent

The whole data set is divided into small batches.

The parameters are updated once for each batch.

```
if optimizer=='MBGD':#mini batch gradient descent  
    ox=np.hstack([X,y.reshape(-1,1)])  
    np.random.shuffle(ox)  
    X,y=ox[:,0:2],ox[:,2]  
    batch_size=5  
    sample_number=int(X.shape[0]/batch_size)  
    er=0  
    for i in range(batch_size):  
        X1=X[i*sample_number:(i+1)*sample_number,:]  
        y1=y[i*sample_number:(i+1)*sample_number]  
        y_hat=self.net_input(X1)  
        y_hat=self.activation(y_hat)  
        self.w=self.w+self.eta*np.matmul((y1-y_hat),X1)  
        self.w0=self.w0+self.eta*sum(y1-y_hat)  
        er=er+sum((y1-y_hat)**2)
```

Stochastic gradient descent

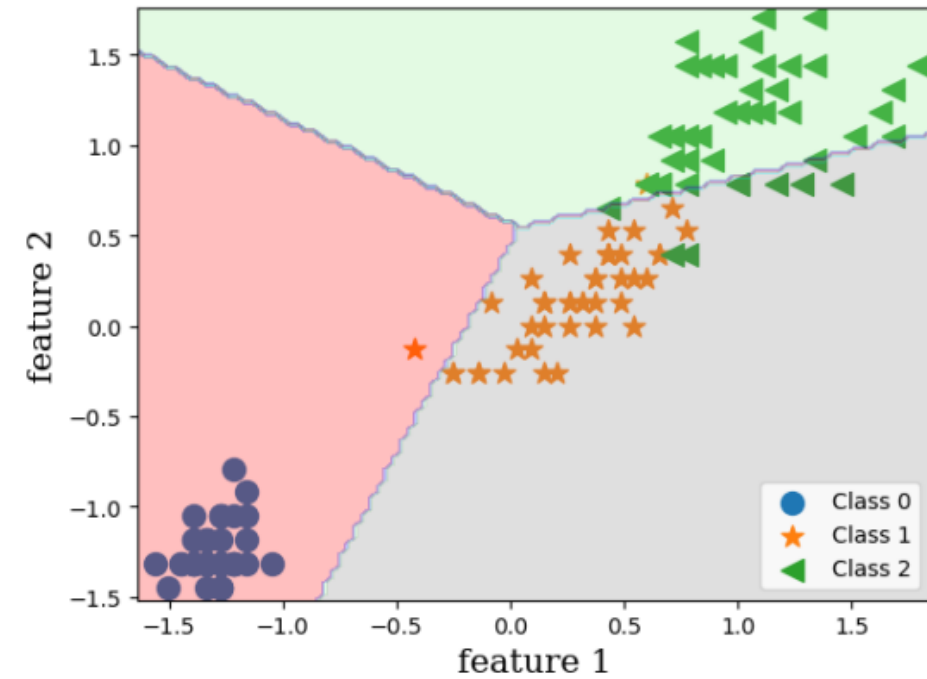
The parameters are updated once for sample.

```
elif optimizer=='SGD':#stochastic gradient descent  
    ox=np.hstack([X,y.reshape(-1,1)])  
    np.random.shuffle(ox)  
    X,y=ox[:,0:2],ox[:,2]  
    batch_size=X.shape[0]  
    sample_number=int(X.shape[0]/batch_size)  
    er=0  
    for i in range(batch_size):  
        X1=X[i*sample_number:(i+1)*sample_number,:]  
        y1=y[i*sample_number:(i+1)*sample_number]  
        y_hat=self.net_input(X1)  
        y_hat=self.activation(y_hat)  
        self.w=self.w+self.eta*np.matmul((y1-y_hat),X1)  
        self.w0=self.w0+self.eta*sum(y1-y_hat)  
        er=er+sum((y1-y_hat)**2)
```

3

A Tour of Machine Learning Classifiers Using scikit-learn

Using scikit-learn for perceptron



Dataset: iris

```
def plot_boundary_graph(X,y,classifier):
    font2 = {'family':'serif','color':'black','size':15}
    label=np.unique(y)
    x1=X[(y==label[0])]
    plt.scatter(x1[:,0],x1[:,1],s=100,marker='o',label=f'Class {label[0]}')
    x1=X[(y==label[1])]
    plt.scatter(x1[:,0],x1[:,1],s=100,marker='*',label=f'Class {label[1]}')
    x1=X[(y==label[2])]
    plt.scatter(x1[:,0],x1[:,1],s=100,marker='<',label=f'Class {label[2]}')
    plt.legend(loc='lower right')
    plt.xlabel('feature 1', fontdict = font2)#('sepal length')
    plt.ylabel('feature 2', fontdict = font2)#('petal length')
    x0=np.arange(min(X[:,0])-0.05*abs(min(X[:,0])),max(X[:,0])+0.05*abs(min(X[:,0])),(max(X[:,0])-min(X[:,0]))/100)
    y0=np.arange(min(X[:,1])-0.05*abs(min(X[:,1])),max(X[:,1])+0.05*abs(min(X[:,1])),(max(X[:,1])-min(X[:,1]))/100)
    x01,y01=np.meshgrid(x0,y0)
    c=np.hstack((x01.reshape(-1,1),y01.reshape(-1,1)))
    c_y=classifier.predict(c)
    plt.contourf(x01,y01,c_y.reshape(x01.shape),alpha=0.25,colors=['red', 'blue', 'lightgreen', 'gray', 'cyan'])
    plt.show()
```

```
import pandas as pd
import numpy as np
import numpy as npexit
import matplotlib.pyplot as plt
df=pd.read_csv(r'F:\learning\ML and DL with Python\ch02\iris.data',header=None)
#Label encoder
from sklearn import preprocessing
le = preprocessing.LabelEncoder()
df[4]=le.fit_transform(df[4])
X,y=df.iloc[:,2:4].values,df.iloc[:,4].values
#train and test set split
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.1,random_state=0,stratify=y)
#normalization
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
scaler.fit(X_train)
X_train=scaler.transform(X_train)
X_test=scaler.transform(X_test)
#train model
from sklearn.linear_model import Perceptron
f=Perceptron(eta0=0.0001)
f.fit(X_train,y_train)
accuracy=f.score(X_test,y_test)
print(accuracy)
plot_boundary_graph(X_train,y_train,f)
```


Logistic regression

Odds ratio: the odds in favor of an event $\frac{p}{1-p} \in (0, +\infty)$

Logit function: $z = \log\left(\frac{p}{1-p}\right) \in (-\infty, +\infty)$

Inverse function: $p = \frac{1}{1+e^{-z}}$ (sigmoid function) $\in (0, 1)$

Because $w_0 + x_1^i w_1 + x_2^i w_2 + \dots + x_m^i w_m \in (-\infty, +\infty)$, it can regard as logit function. Then we get

$$z^i = w_0 + x_1^i w_1 + x_2^i w_2 + \dots + x_m^i w_m = \log\left(\frac{p^i}{1-p^i}\right)$$

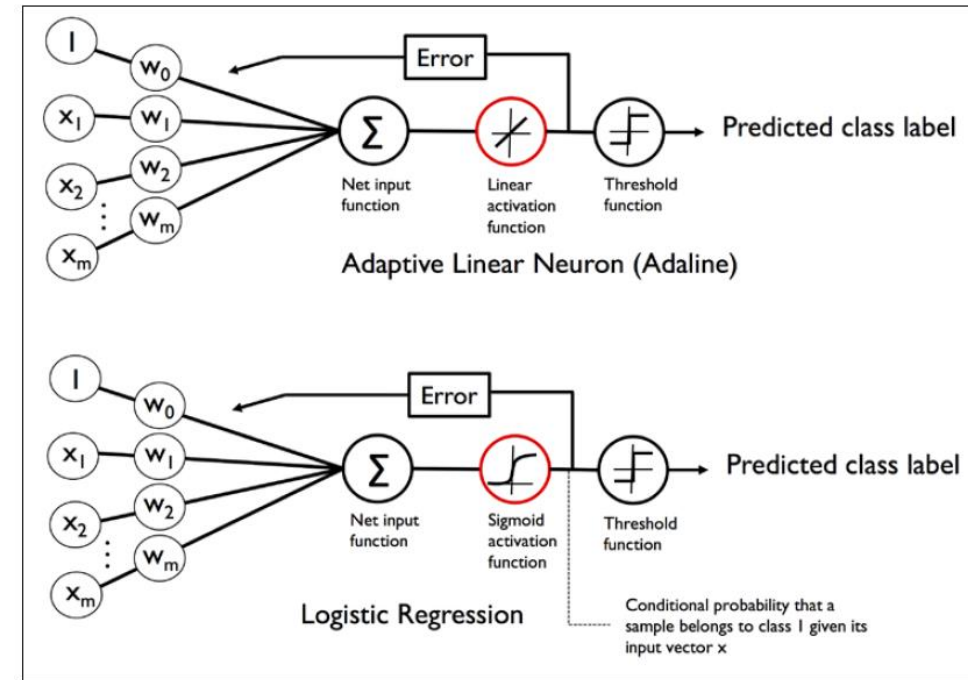
$$p^i = \frac{1}{1+e^{-z^i}} = \frac{1}{1+e^{-(w_0 + x_1^i w_1 + x_2^i w_2 + \dots + x_m^i w_m)}} = \varphi(z^i)$$

Here $p^i = p(\text{class} = 1 | (x_1^i, x_2^i, \dots, x_m^i))$

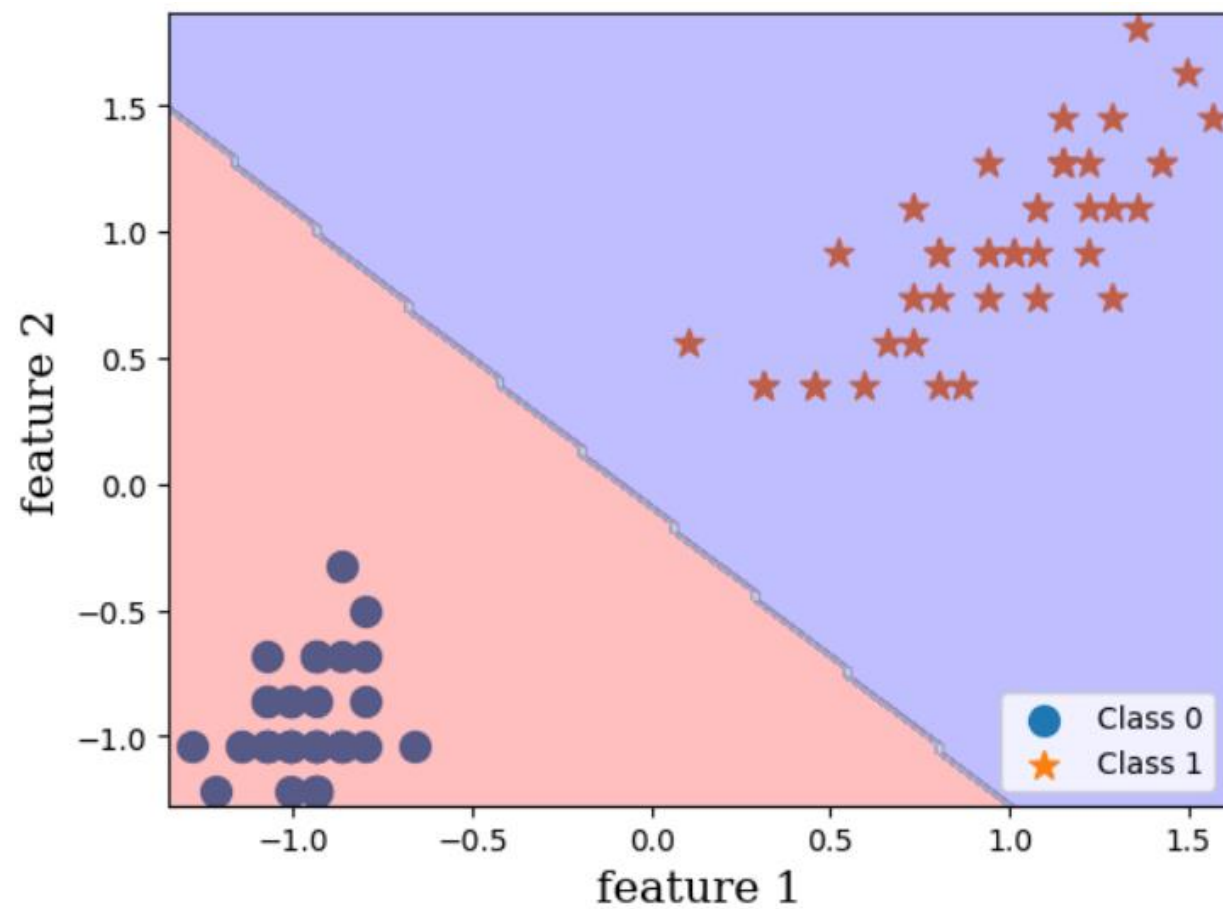
$$\text{Likelihood} = \prod_{i=1}^n (p^i)^{y^i} (1 - p^i)^{(1-y^i)}$$

Loss function (likelihood):

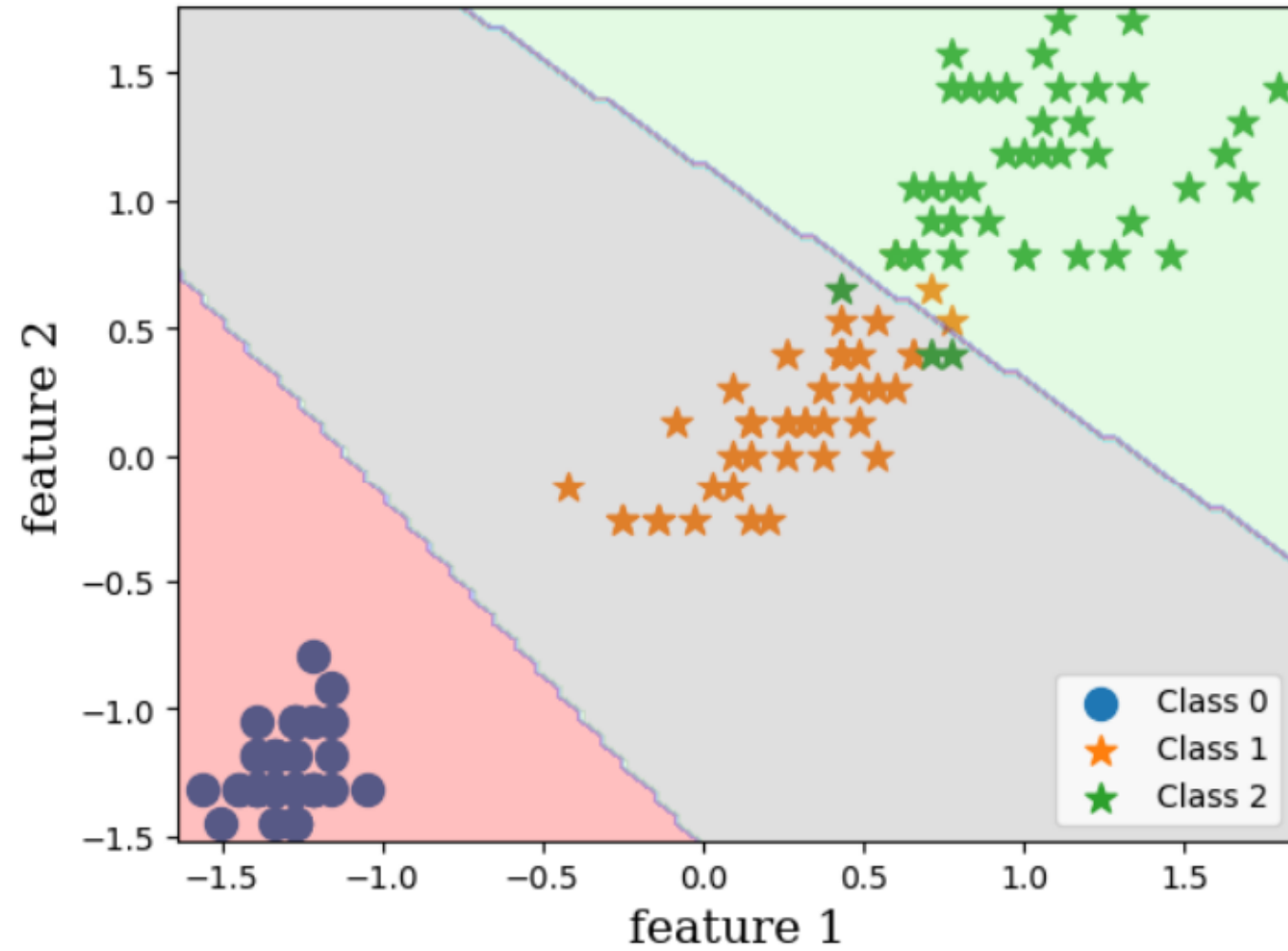
$$L = - \sum_{i=1}^n (y^i \log(p^i) + (1 - y^i) \log(1 - p^i))$$



Logistic Regression from scratch



Logistic regression from sklearn



Regularization

Tackle overfitting via regularization



Variance measures the consistency (or variability) of the model prediction for a particular sample instance if we were to retrain the model multiple times, for example, on different subsets of the training dataset. We can say that the model is sensitive to the randomness in the training data. In contrast, bias measures how far off the predictions are from the correct values in general if we rebuild the model multiple times on different training datasets; bias is the measure of the systematic error that is not due to randomness.

In general, as we increase the number of tunable parameters in a model, it becomes more flexible, and can better fit a training data set. It is said to have lower error, or [bias](#). However, for more flexible models, there will tend to be greater **variance** to the model fit each time we take a set of [samples](#) to create a new training data set. It is said that there is greater [variance](#) in the model's [estimated parameters](#).

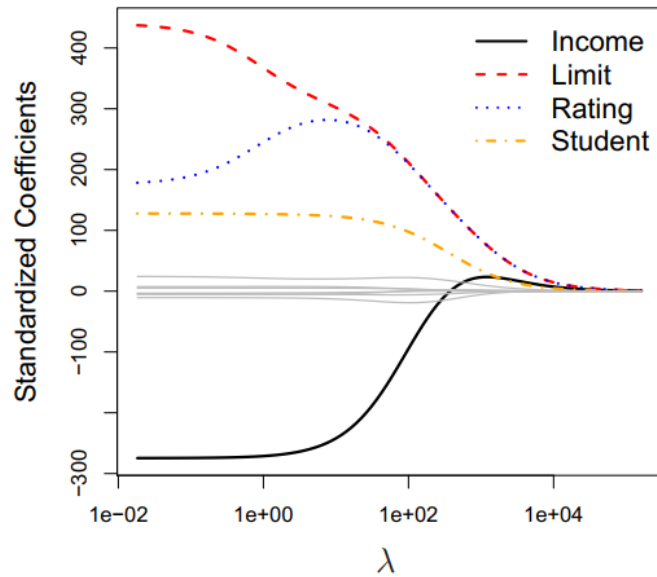
Variance: If we retrain the model multiple times on different subset of the training set, we can get the variance of fitted model (parameter), or the variance of the model prediction for a particular sample

Bias, Variance

L1 regularization, L2 regularization

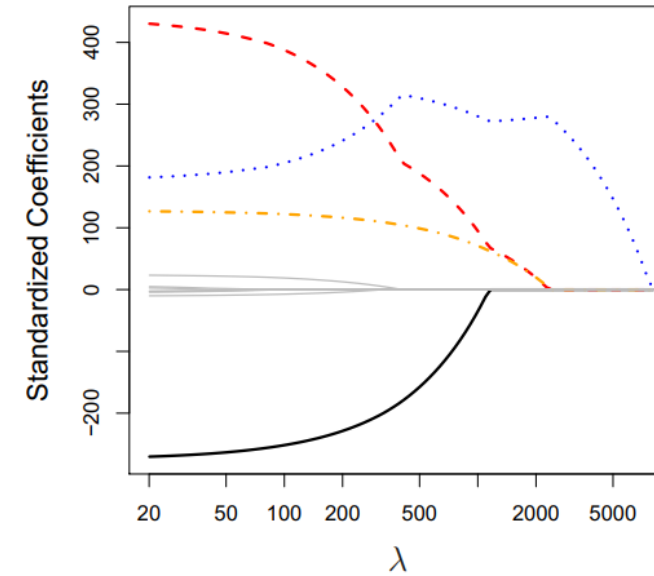
Ridge Regression (L2)

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (6.5)$$



Lasso Regression (L1)

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \quad (6.7)$$



Note that in (6.5), the shrinkage penalty is applied to $\beta_1, \dots, \beta_p$, but not to the intercept β_0 . We want to shrink the estimated association of each variable with the response; however, we do not want to shrink the intercept, which is simply a measure of the mean value of the response when $x_{i1} = x_{i2} = \dots = x_{ip} = 0$. If we assume that the variables—that is,

Another Formulation for Ridge Regression and the Lasso

One can show that the lasso and ridge regression coefficient estimates solve the problems

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p |\beta_j| \leq s \quad (6.8)$$

and

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p \beta_j^2 \leq s, \quad (6.9)$$

respectively. In other words, for every value of λ , there is some s such that the Equations (6.7) and (6.8) will give the same lasso coefficient estimates. Similarly, for every value of λ there is a corresponding s such that Equations (6.5) and (6.9) will give the same ridge regression coefficient estimates.

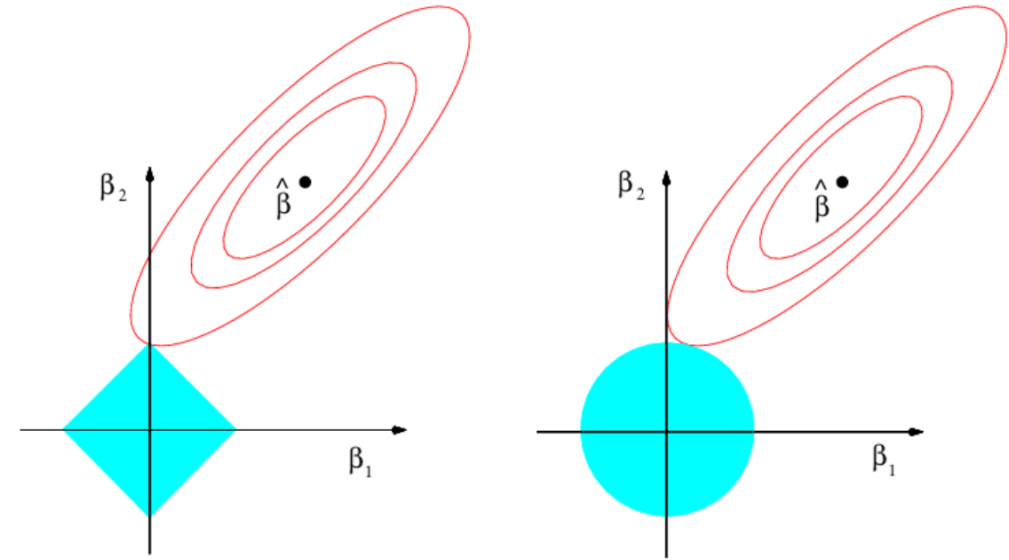
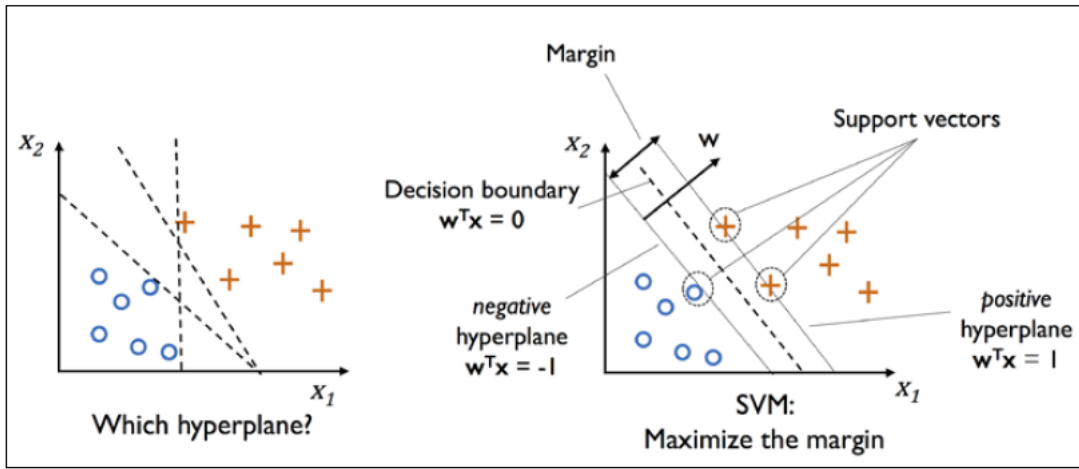


FIGURE 6.7. Contours of the error and constraint functions for the lasso (left) and ridge regression (right). The solid blue areas are the constraint regions, $|\beta_1| + |\beta_2| \leq s$ and $\beta_1^2 + \beta_2^2 \leq s$, while the red ellipses are the contours of the RSS.

Support Vector Machine

SVM



The distance between two hyperplanes: $\frac{2}{|W|}$

Loss function: $L = \frac{1}{2} W$

Constraint: $y^i (w_0 + W^T X^i) \gg 1, \forall_i$

How to get W: quadratic programming

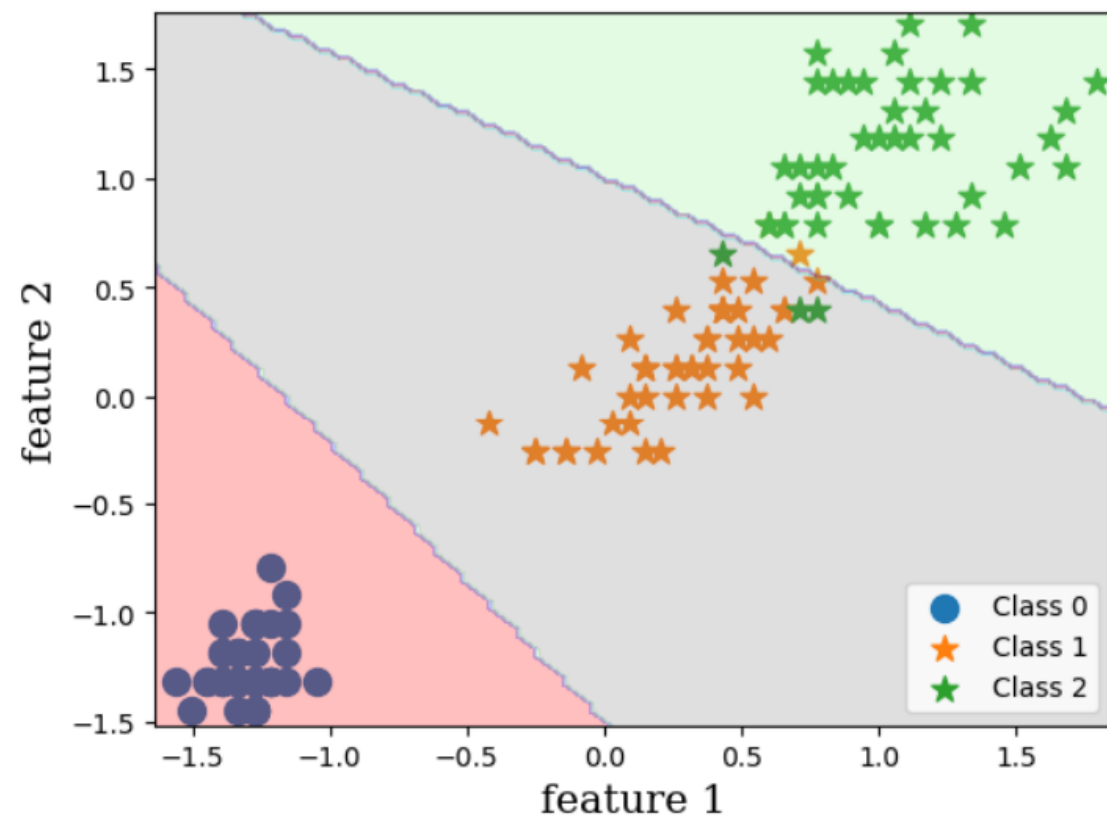
Soft-margin classification

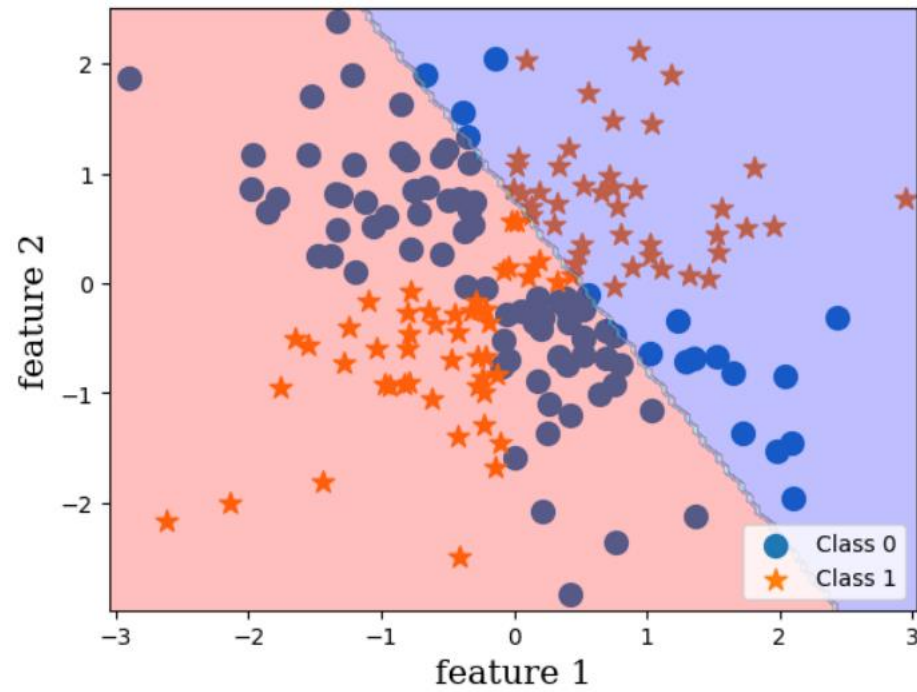
Loss function: $L = \frac{1}{2} W + C \sum_{i=1}^n \varepsilon^i$

Constraint: $y^i (w_0 + W^T X^i) \gg 1 - \varepsilon^i, \forall_i$

How to get W: quadratic programming

ε^i is Slack variable





Can not work for data that is
non-linear separable.

**Using the kernel trick to find separating
hyperplanes in high-dimensional space**

Linear Machine Learning Techniques Using Only Dot Products (Detailed Explanation)

Many linear machine learning algorithms can be expressed purely in terms of **dot products** between data points. This property is crucial because it enables **kernelization**, allowing these methods to handle nonlinear problems efficiently. Below, we explore this concept in detail, covering:

- 1. Why Dot Products Matter
- 2. Key Linear Algorithms & Their Dot Product Forms
- 3. The Kernel Trick & Nonlinear Extensions
- 4. Mathematical Justifications

1. Why Dot Products Matter

Linear models typically rely on computations of the form:

$$f(x) = w^T x + b$$

where:

- w is the weight vector,
- x is the input feature vector,
- b is the bias term.

The key observation is that w **is often a linear combination of training samples** (due to representer theorems), meaning:

$$w = \sum_{i=1}^N \alpha_i x_i$$

Thus, predictions can be written as:

$$f(x) = \sum_{i=1}^N \alpha_i x_i^T x + b$$

This shows that **only dot products $\langle x_i, x \rangle$ are needed**, not the explicit feature vectors x_i .

2. Key Linear Algorithms & Their Dot Product Forms

(A) Linear Regression (Ordinary Least Squares, OLS)

- Standard form:** Minimizes $\|y - Xw\|^2$, solution: $w = (X^T X)^{-1} X^T y$.
- Dot product form:**
 - The Gram matrix $G = X X^T$ contains all pairwise dot products $G_{ij} = x_i^T x_j$.
 - The solution can be rewritten in dual form using G .

(B) Ridge Regression (L2-Regularized Linear Regression)

- Primal form:** Minimizes $\|y - Xw\|^2 + \lambda \|w\|^2$.
- Dual form:** Solution can be written as:

$$w = \sum_{i=1}^N \alpha_i x_i \quad \text{where} \quad \alpha = (G + \lambda I)^{-1} y$$

(Again, $G = X X^T$ is the Gram matrix.)

(C) Support Vector Machines (SVM)

- Primal form:** Minimizes $\frac{1}{2} \|w\|^2 + C \sum \xi_i$ subject to $y_i(w^T x_i + b) \geq 1 - \xi_i$.
- Dual form:**

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^T x_j$$

- Depends **only on dot products** $x_i^T x_j$.

(D) Principal Component Analysis (PCA)

- Computes eigenvectors of the covariance matrix $X^T X$.
- The covariance matrix is a sum of outer products $X^T X = \sum x_i x_i^T$.
- Kernel PCA replaces $x_i^T x_j$ with $K(x_i, x_j)$.

(E) Logistic Regression

- Decision function: $\sigma(w^T x + b)$, where σ is the sigmoid.
- If w is expressed as $\sum \alpha_i x_i$, predictions depend only on $x_i^T x$.

(F) Perceptron Algorithm

- Updates weights via $w \leftarrow w + \eta(y_i - \text{sign}(w^T x_i))x_i$.
- At test time, only $w^T x = \sum \alpha_i x_i^T x$ is needed.

3. The Kernel Trick: Extending to Nonlinear Problems

Since these algorithms depend only on dot products, we can replace $\langle x_i, x_j \rangle$ with a **kernel function** $K(x_i, x_j)$, which implicitly computes dot products in a higher-dimensional space.

Common Kernel Functions

- **Linear Kernel:** $K(x, z) = x^T z$ (standard dot product).
- **Polynomial Kernel:** $K(x, z) = (x^T z + c)^d$.
- **Gaussian (RBF) Kernel:** $K(x, z) = \exp(-\gamma \|x - z\|^2)$.

Example: Kernelized Ridge Regression

- Replace XX^T with the kernel matrix K , where $K_{ij} = K(x_i, x_j)$.
- The solution becomes:

$$\alpha = (K + \lambda I)^{-1} y$$

- Predictions:

$$f(x) = \sum_{i=1}^N \alpha_i K(x_i, x) + b$$

Example: Kernel SVM

- The dual SVM optimization becomes:

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

- Decision function:

$$f(x) = \sum_{i=1}^N \alpha_i y_i K(x_i, x) + b$$

4. Mathematical Justification: The Representer Theorem

A key theoretical foundation is the **Representer Theorem**, which states that for many regularization problems of the form:

$$\min_w \sum_{i=1}^N L(y_i, w^T x_i) + \lambda \|w\|^2$$

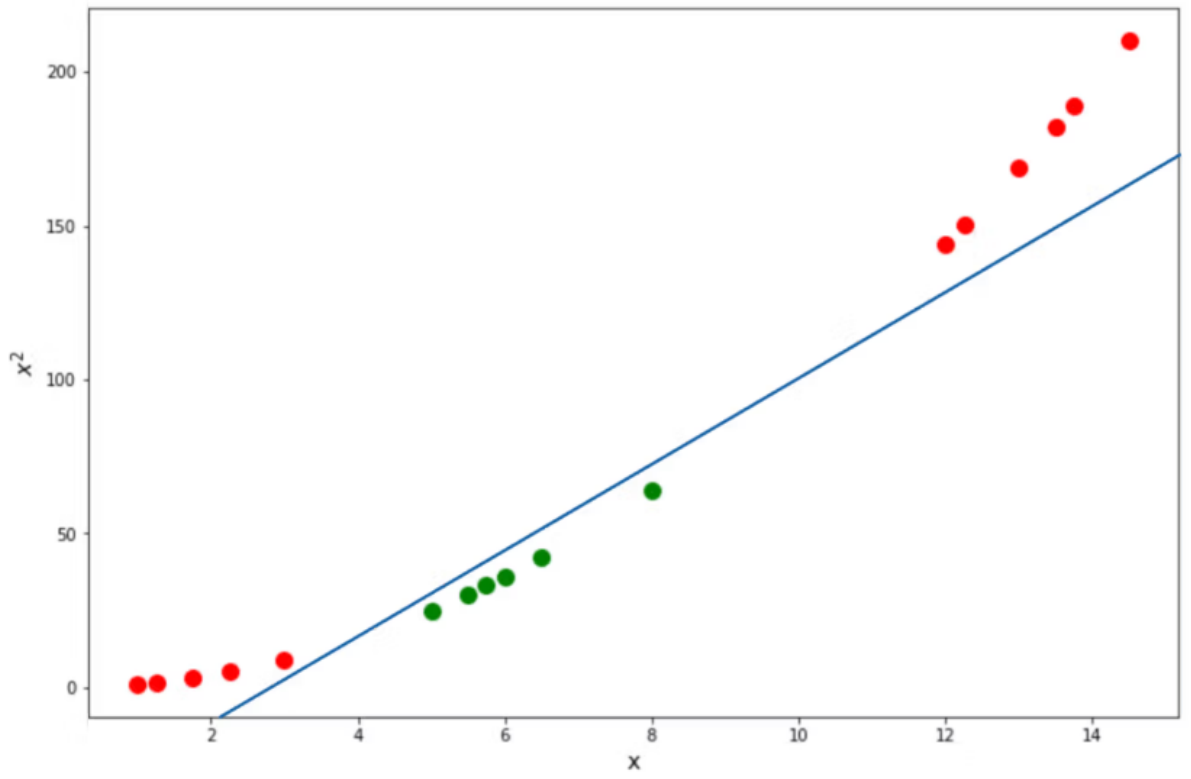
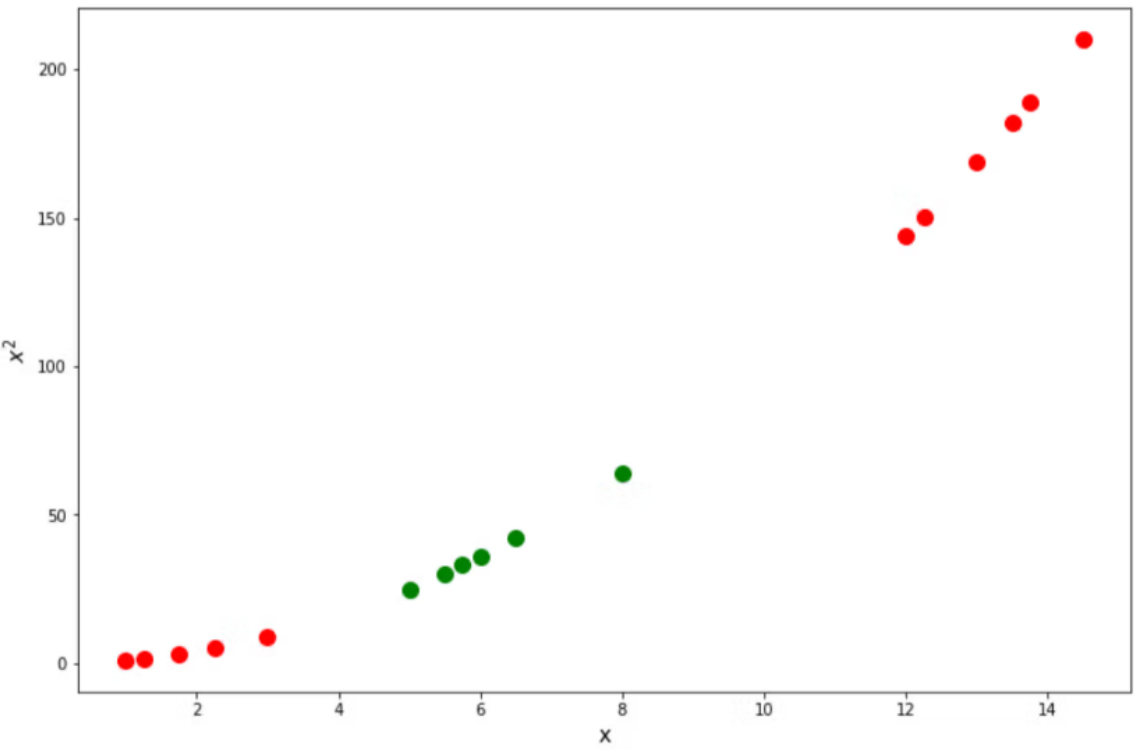
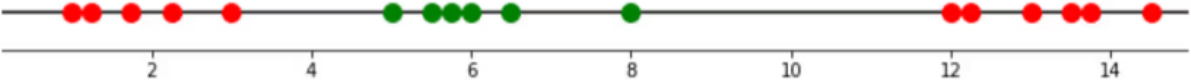
the optimal solution w can be written as:

$$w = \sum_{i=1}^N \alpha_i x_i$$

This means w **is a linear combination of training points**, so predictions depend only on dot products.

Summary & Implications

- **Dot products are sufficient** for many linear ML models.
- **Kernel trick** generalizes them to nonlinear cases by replacing $x_i^T x_j$ with $K(x_i, x_j)$.
- **Key algorithms** (SVM, Ridge Regression, PCA, etc.) can be "kernelized."
- **Efficiency:** Kernel methods avoid explicitly computing high-dimensional features, working only with the kernel matrix.



Introduction

Suppose you have an N-dimensional dataset, and you would like to apply some linear machine learning technique to it, e.g. find a linear separating hyperplane - but it's not working, because the shape of the dataset is too non-linear.

One way to go is to try finding a non-linear separator, e.g. stop looking for hyperplanes and start looking for higher-order surfaces. We're not interested in this option because linear is simpler.

Another way to go is to transform your input variables so that the shape of the dataset becomes more linear. E.g. if there's a clear parabola in the shape, you might want to square one of the variables to linearise it.

Transforming the dataset

Note that you don't have to preserve the dimensionality of the original dataset when doing this transformation! E.g. all you need to do for looking a polynomial hyperplane of order 3 separating a 2-dimensional dataset is to map each point (x, y) to the 6-dimensional vector $(x, x^2, x^3, y, y^2, y^3)$. The amount of information is the same, but now even a linear classifier can make use of polynomial trends; and you can easily train a linear classifier on the transformed dataset, which will give you a non-linear classifier on the original dataset.

The kernel trick

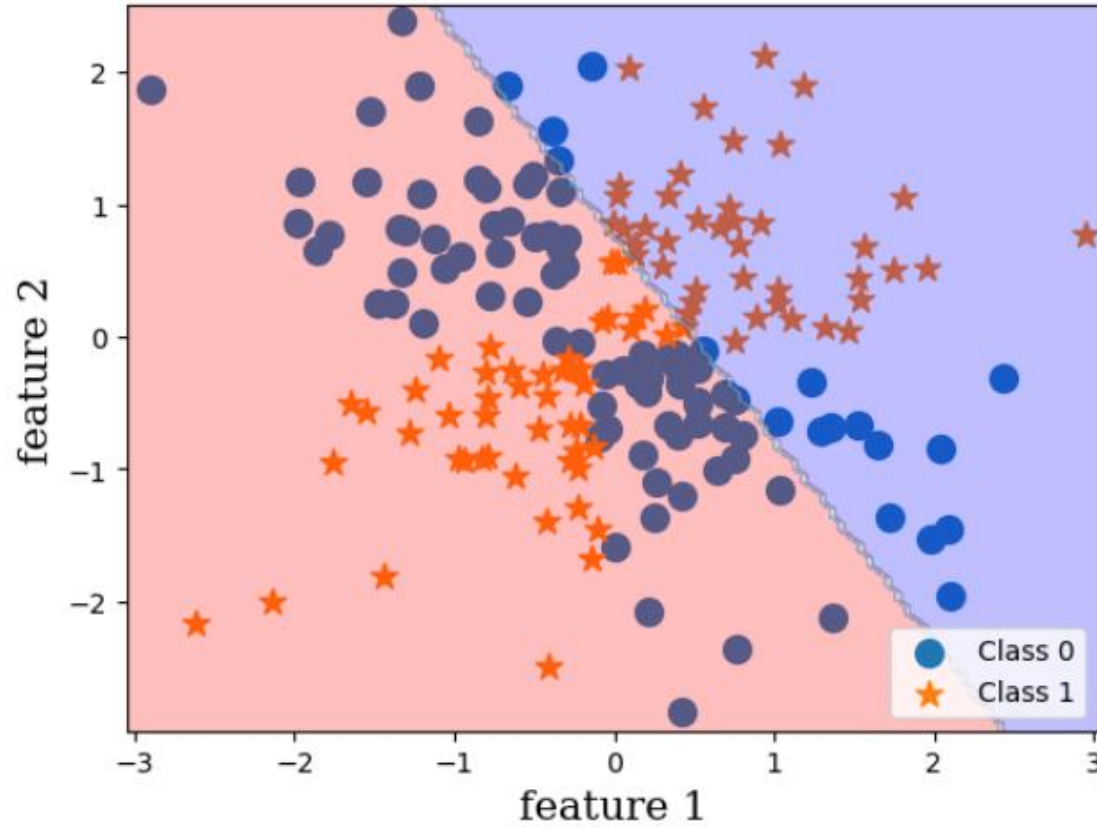
Let's call your transformation function F . Most linear machine learning techniques can be implemented with using only the dot product operation, call it P . If you can compute $a \cdot b$ for any two points of your dataset, you often don't need anything else - you don't need to even know the dimensionality of the points.

So what if you knew a function $K(a, b)$ such that $K(a, b) = F(a) \cdot F(b)$. Then, during learning, every time you needed to compute $F(a) \cdot F(b)$, you'd just compute $K(a, b)$ - you wouldn't need to actually apply the function F - the transformation would exist only "implicitly". K is called the kernel.

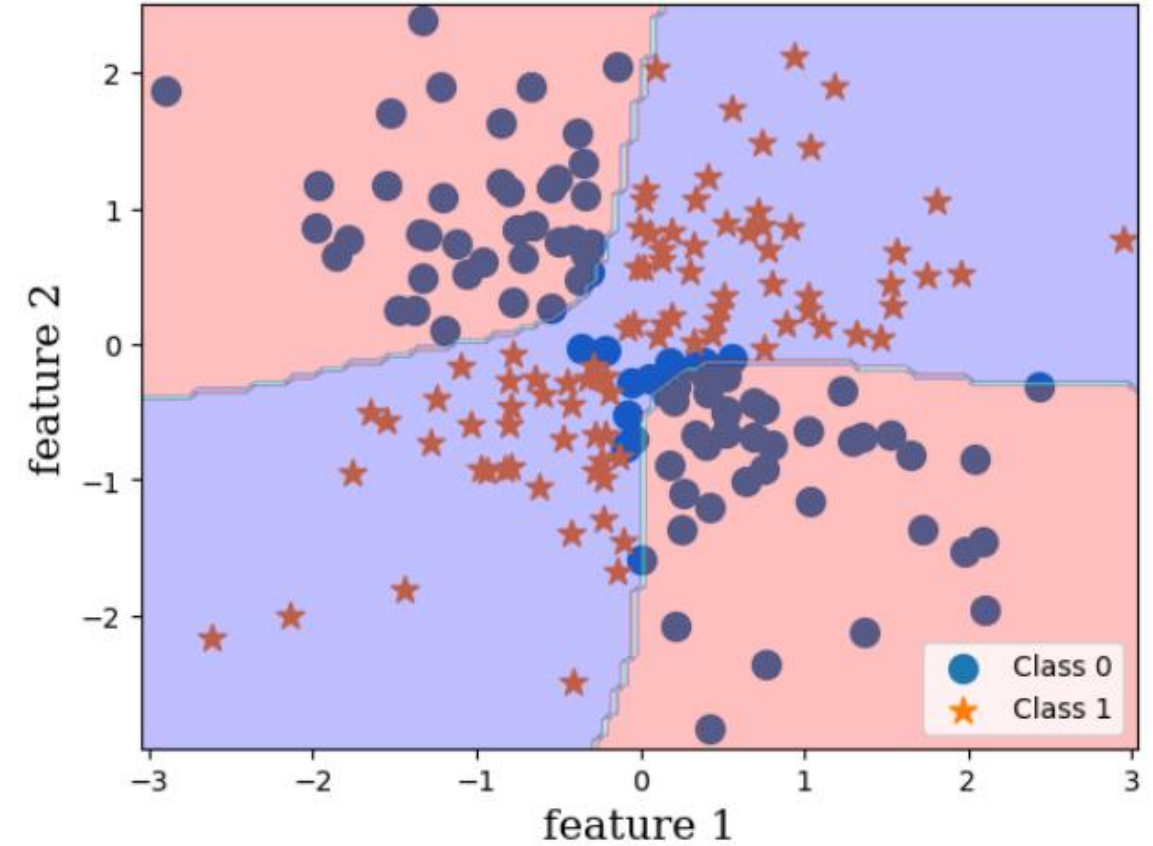
Magic of the kernel trick: transformed dataset can be implicit

This opens up quite a number of possibilities. For example, you no longer need the transformed dataset to be small-dimensional (fit in memory), or even finite-dimensional, or even to exist at all - you only need the function K to obey a number of dot-product-like properties. With some complicated enough K functions, you may get arbitrarily precise separation - the only danger is overfitting.

```
from sklearn import svm
f=svm.SVC(kernel="linear")
f.fit(X_train,y_train)
accuracy=f.score(X_test,y_test)
print(accuracy)
plot_boundary_graph(X_train,y_train,f)
```



```
from sklearn import svm
f=svm.SVC(kernel="rbf")
f.fit(X_train,y_train)
accuracy=f.score(X_test,y_test)
print(accuracy)
plot_boundary_graph(X_train,y_train,f)
```



Decision tree learning